

이주영

Frontend Developer

juyung0903@naver.com

[GitHub](#) | [Portfolio](#)

Summary

React, Next.js, React Native 기반으로 실시간 통신, 캐싱 전략, 모바일 위치 기반 기능을 구현해온 신입 프론트엔드 개발자입니다.

React Profiler로 렌더링 병목을 분석해 특정 화면 렌더링 시간을 211ms -> 72ms로 약 65% 단축했고, React Query 캐시 직접 갱신 구조를 적용해 댓글 이벤트 처리 시 refetch 요청을 10회 -> 0회로 제거하고 처리 시간을 7854ms -> 1248ms로 단축했습니다.

단순 기능 구현보다 API 요청 흐름, 상태 동기화, 실시간 연결 안정성, 사용자 대기 경험을 함께 개선하는 방식으로 프로젝트를 진행했습니다.

Skills

Category	Skills
Frontend	TypeScript, React, Next.js, React Native, Tailwind CSS
State / Data	TanStack Query, Zustand, React Hook Form, Zod
Realtime / Mobile	Socket.io, WebSocket (STOMP), Expo, Expo Router, EAS Update, FCM, PWA, Google Maps API, React Native Maps

Projects

CodeMate | AI 기반 코드 리뷰 및 실시간 협업 플랫폼

개인 프로젝트 | 2026.02 - 2026.04

[GitHub](#)

Next.js App Router 기반으로 GitHub PR 분석, AI 코드 리뷰 요청, 실시간 댓글/알림 동기화를 제공하는 개발자용 플랫폼을 구현했습니다.

- **문제:** 댓글/알림 데이터가 refetch 기반으로 동작해 이벤트 10회 수신 시 API 요청도 10회 반복되고, UI 반영 지연이 발생했습니다.
해결: Socket.io 이벤트 수신 시 TanStack Query의 setQueryData로 캐시를 직접 갱신하는 구조로 전환했습니다.
결과: 댓글 이벤트 10회 기준 API 요청 10회 -> 0회, 처리 시간 7854ms -> 1248ms로 약 84.1% 단축했습니다.
- **문제:** 컴포넌트 단위로 WebSocket 연결과 이벤트 핸들러가 분산되어 중복 연결 가능성과 이벤트 흐름 추적 비용이 커졌습니다.
해결: useSyncExternalStore 기반 단일 소켓 구독 구조로 통합하고, 연결/재연결/구독 책임을 한 계층으로 모았습니다.
결과: WebSocket 연결 경로를 단일화하고, 실시간 댓글/알림 이벤트 흐름을 컴포넌트 외부에서 추적할 수 있도록 정리했습니다.
- **문제:** AI 코드 리뷰 요청 후 분석 완료까지 화면 전환과 결과 확인이 지연되어 사용자가 처리 상태를 알기 어려웠습니다.
해결: 리뷰 요청 즉시 PENDING 상태를 반환하고, 비동기 분석 완료 후 알림과 재조회로 결과를 갱신하는 흐름을 적용했습니다.
결과: AI 분석 대기 중에도 사용자에게 요청 접수 상태를 즉시 보여주고, 완료 시점에 결과 확인 흐름으로 연결되도록 개선했습니다.
- **문제:** 통계 대시보드에서 여러 API가 순차 호출되어 초기 데이터 대기 시간이 길어졌습니다.
해결: 통계 API를 Promise.all로 병렬 호출하고, 차트 컴포넌트를 next/dynamic으로 분리했습니다.
결과: waterfall 요청 구조를 제거하고, 초기 렌더링 비용과 데이터 대기 시간을 분리해 관리할 수 있도록 개선했습니다.

Career Hub | 취업 지원 현황 관리 플랫폼

팀 프로젝트 (FE 2 / BE 2) | 2025.11 - 2026.02

[GitHub](#)

Next.js 기반으로 지원 현황, 전형 일정, 알림을 통합 관리하는 PWA 서비스를 개발했습니다.

- **문제:** 전형 일정과 상세 정보 수정 후 전체 상세 조회가 반복되어 불필요한 네트워크 요청이 발생했습니다.
해결: 백엔드와 수정 API의 책임 범위를 조율하고, 수정 응답 및 입력값을 기준으로 TanStack Query 캐시를 부분 갱신했습니다.
결과: 수정 후 추가 상세 조회 3회 -> 0회로 줄이고, 사용자 입력 결과가 UI에 즉시 반영되도록 개선했습니다.
협업: 백엔드와 응답 스키마, 수정 책임 단위, 프론트 캐시 갱신 기준을 맞춰 API 호출 흐름을 정리했습니다.
- **문제:** 전형 결과 변경 시 현재 단계와 최종 상태가 어긋나는 상태 불일치가 발생했습니다.
해결: 단계별 상태 계산 로직과 UI 표시 상태를 분리하고, 팀 내에서 상태 전이 기준을 정리했습니다.
결과: 상태 계산 책임을 명확히 분리해 같은 데이터에서 화면별 상태가 다르게 표시되는 문제를 제거했습니다.
협업: 백엔드와 상태 enum 기준, 최종 상태 계산 책임, 예외 케이스 처리 방식을 조율했습니다.
- **문제:** 브라우저별 Web Push 지원 차이로 알림 수신이 불안정했습니다.
해결: Service Worker, FCM 연동, iOS Safari 분기 처리를 적용하고 백엔드 알림 발송 흐름과 연동했습니다.
결과: 브라우저별 지원 차이를 분기 처리해 주요 환경에서 푸시 수신 흐름이 동작하도록 구현했습니다.
- **문제:** 네트워크가 불안정한 환경에서 지원 현황 화면이 빈 화면으로 보이는 문제가 있었습니다.
해결: React Query 캐싱 전략을 적용해 기존 데이터를 유지하고, 재검증 중에도 이전 상태를 확인할 수 있도록 처리했습니다.
결과: 일시적인 네트워크 지연 상황에서도 사용자가 기존 지원 현황을 확인할 수 있도록 개선했습니다.

Fly:On | 체험 비행 일정 관리 앱

팀 프로젝트 (FE 2 / BE 2 / Design 1) | 2025.06 - 2025.11

[GitHub](#)

React Native(Expo) 기반으로 체험 비행 일정 생성, 일정 관리, 드래그 인터랙션을 제공하는 모바일 앱을 개발했습니다.

- **문제:** 특정 일정 화면에서 렌더링 지연으로 프레임 드롭이 발생했습니다.
해결: React Profiler의 commit time 기준으로 병목 컴포넌트를 식별하고, 렌더링 범위와 컴포넌트 구조를 조정했습니다.
결과: 렌더링 시간 211ms -> 72ms로 약 65% 단축하고, 프레임 드롭을 약 80% 줄였습니다.
- **문제:** Drag & Drop UI에서 드래그 카드가 다른 UI 아래로 가려지는 레이어 충돌이 발생했습니다.
해결: PanResponder 기반 커스텀 Drag & Drop과 Portal 패턴을 적용해 드래그 중인 요소의 렌더링 계층을 분리했습니다.
결과: 드래그 카드가 화면 요소에 가려지는 문제를 해결하고, 일정 조정 인터랙션을 안정화했습니다.
- **문제:** 외부 AI 응답 지연이나 실패 상황에서 사용자가 장애 상태를 늦게 인지하는 문제가 있었습니다.
해결: Timeout, Error Boundary, 재시도 가능한 Fallback UI를 설계했습니다.
결과: 장애 감지 시간을 10초 이내로 제한하고, 실패 후 재시도 가능한 흐름을 제공했습니다.
- **협업:** 디자이너와 드래그 인터랙션의 터치 영역, 레이어 우선순위, 실패 상태 UI를 조율하고, 백엔드와 일정 생성/수정 API 응답 구조를 맞췄습니다.

러너스하이 | 위치 기반 실시간 러닝 앱

졸업작품 (FE 1 / BE 2) | 2025.02 - 2025.07

[GitHub](#)

React Native(Expo), TypeScript, Zustand, TanStack Query, WebSocket(STOMP), expo-location, React Native Maps 기반으로 실시간 위치 추적, 코스 탐색, 솔로/경쟁/크루 러닝을 지원하는 모바일 러닝 앱 프론트엔드를 개발했습니다.

- **문제:** GPS 신호 편차로 거리와 페이스 계산 값이 흔들렸습니다.
해결: expo-location 수집 주기와 거리 누적 로직을 조정하고, 위치 업데이트 흐름을 러닝 모드별로 분리했습니다.
결과: 거리/페이스 계산 로직을 러닝 기록 저장 흐름과 분리해 GPS 편차가 화면 표시와 기록 저장에 미치는 영향을 줄였습니다.
- **문제:** 네트워크 오류나 일시적 연결 끊김이 발생하면 실시간 위치 동기화가 중단됐습니다.
해결: heartbeatIncoming, heartbeatOutgoing, reconnectDelay를 적용한 STOMP 클라이언트와 모드별 publish/subscribe 분기 구조를 구현했습니다.
결과: 주행 중 WebSocket 연결이 끊겼을 때 재연결을 시도하고, 모드별 위치 전송 흐름을 분리해 데이터 흐름을 안정화했습니다.
- **문제:** 경쟁자 비교 기능에서 매 초 경쟁자 진행도를 수신해 네트워크 요청과 클라이언트 처리 비용이 커졌습니다.
해결: 백엔드와 경쟁자 기록 전달 방식을 시계열 조회 구조로 조정하고, 프론트엔드에서 현재 러너의 진행도와 비교하도록 변경했습니다.
결과: 매 초 수신 구조를 제거해 불필요한 네트워크 요청을 줄이고, 비교 계산 책임을 클라이언트에서 처리하도록 정리했습니다.
- **문제:** 솔로 러닝, 코스 경쟁, 크루 러닝이 한 흐름에 섞여 상태 전이와 UI 분기가 복잡했습니다.
해결: 러닝 모드별 로직을 커스텀 훅 단위로 분리하고, Zustand 스토어에서 위치/러닝/코스 상태 책임을 나눴습니다.
결과: 모드별 상태 전이와 UI 분기를 분리해 유지보수성과 기능 확장성을 높였습니다.
- **협업:** 백엔드와 STOMP publish/subscribe 경로, 위치 데이터 payload, 경쟁자 기록 조회 방식을 조율했습니다.

Education

한국공학대학교 (졸업 예정)
IT경영 전공 / 소프트웨어학과 복수전공
2019.03 - 2026.02